

---

**Desenvolvimento de Aplicações Web**  
**Lab 06 - Estruturas de Dados em JavaScript**

Ano Letivo: 2025/2026

---

## **1 Introdução**

No Lab 05, focámo-nos na **manipulação do DOM** usando arrays simples (de strings). Agora vamos aprofundar as **estruturas de dados** em JavaScript.

Vamos aprender:

- Como criar e manipular **objetos** (objects)
- Trabalhar com **arrays de objetos**
- Serializar e deserializar com **JSON**
- Guardar e carregar dados persistentemente
- Aplicar tudo isto ao projeto da Lista de Compras

Neste laboratório, vamos transformar a Lista de Compras do Lab 05 numa versão mais poderosa, com quantidades, categorias e persistência de dados.

Na Figura 1 pode ver o aspeto final esperado do trabalho.

The image shows a web application interface for a shopping list. At the top, there's a title 'Lista de Compras'. Below it, a form to add items with a text input 'Adicionar item ...', a dropdown menu set to 'Geral', and an 'Adicionar' button. The list contains three items: 'Leite [ Laticínios]' with a quantity of 6, 'Pão [ Padaria]' with a quantity of 1 and checked, and 'Morangos [ Frutas]' with a quantity of 1. Each item has minus and plus buttons for quantity adjustment and a red 'Remover' button. Summary statistics show 'Total de itens: 3', 'Total de unidades: 8', and 'Itens comprados: 1'. At the bottom, there are buttons for sorting ('Ordenar A-Z', 'Ordenar por categoria', 'Ordenar por quantidade') and a red 'Limpar Tudo' button, along with a 'Modo Escuro' toggle switch.

**Figura 1:** *Aspetto final esperado da Lista de Compras no Lab 06*

## 2 Objetivos

Neste laboratório iremos:

- Compreender objetos em JavaScript (object literals)
- Criar e manipular arrays de objetos
- Usar `map()`, `filter()`, `find()`, `sort()` em arrays
- Entender JSON e as funções `stringify()` e `parse()`
- Implementar persistência com `LocalStorage`
- Aprofundar a organização de código JavaScript

## 3 Pré-requisitos

- Lab 05 concluído (manipulação do DOM)

- Visual Studio Code instalado
- Browser moderno com ferramentas de desenvolvimento
- Compreensão de arrays e funções JavaScript
- Ficheiro lista-compras.html do Lab 05

## 4 Conceitos-Chave

### 4.1 Objetos em JavaScript

Um **objeto** é uma coleção de pares chave-valor. É fundamental em JavaScript:

```
// Criar objeto vazio
let pessoa = {};
console.log(pessoa);

// Preencher objeto com propriedades
pessoa = {
  nome: "Maria",
  idade: 25,
  estudante: true
};

// Aceder a propriedades
console.log(pessoa.nome);      // "Maria"
console.log(pessoa['idade']);  // 25

// Modificar propriedades
pessoa.idade = 26;
pessoa['estudante'] = false;

// Adicionar novas propriedades
pessoa.cidade = "Lisboa";

// Remover propriedades
delete pessoa.cidade;
```

### 4.2 Métodos de Objetos

Objetos podem ter **métodos** (funções dentro de objetos):

```
let pessoa = {
  nome: "João",
  saudar: function() {
    return "Olá, " + this.nome;
  }
};

console.log(pessoa.saudar()); // "Olá, João"
```

**Nota:** `this` refere-se ao próprio objeto.

### 4.3 Arrays de Objetos

Combinar arrays com objetos é muito poderoso:

```
// Array de objetos
let compras = [
  { id: 1, nome: "Leite", quantidade: 2, categoria: "Laticínios" },
  { id: 2, nome: "Pão", quantidade: 1, categoria: "Padaria" },
  { id: 3, nome: "Maçã", quantidade: 5, categoria: "Frutas" }
];

// Percorrer
compras.forEach(item => {
  console.log(item.nome);
});

// Filtrar (frutas e legumes)
let frutas = compras.filter(item => item.categoria === "Frutas");

// Transformar (só nomes)
let nomes = compras.map(item => item.nome);

// Encontrar
let leite = compras.find(item => item.nome === "Leite");

// Verificar se existe
let temPao = compras.some(item => item.nome === "Pão");
```

### 4.4 JSON (JavaScript Object Notation)

JSON é um formato de texto para representar dados estruturados:

```
// Objeto JavaScript
let pessoa = {
  nome: "Ana",
  idade: 30
};

// Converter para JSON (string)
let jsonString = JSON.stringify(pessoa);
// Resultado: '{"nome":"Ana","idade":30}'

// Converter de volta para objeto
let pessoaRecuperada = JSON.parse(jsonString);
console.log(pessoaRecuperada.nome); // "Ana"
```

#### JSON vs Objeto

**Objeto JavaScript** é código executável. **JSON** é apenas um formato de texto.  
JSON é essencial para: - Guardar dados em ficheiros ou bases de dados - Enviar dados pela internet - Persistência com LocalStorage

### 4.5 LocalStorage

**LocalStorage** permite guardar dados persistentemente no browser:

```
// Guardar dados
let dados = { nome: "Maria", idade: 25 };
localStorage.setItem('pessoa', JSON.stringify(dados));

// Recuperar dados
let jsonRecuperado = localStorage.getItem('pessoa');
let pessoaRecuperada = JSON.parse(jsonRecuperado);

// Remover dados
localStorage.removeItem('pessoa');

// Limpar tudo
localStorage.clear();
```

**Importante:** LocalStorage funciona como um dicionário de strings. Por isso convertemos com JSON.

## 4.6 Métodos Úteis de Arrays

| Método         | Descrição                                      |
|----------------|------------------------------------------------|
| push(item)     | Adiciona ao final                              |
| pop()          | Remove e retorna o último                      |
| filter(func)   | Filtra elementos (retorna novo array)          |
| map(func)      | Transforma elementos (retorna novo array)      |
| find(func)     | Encontra primeiro elemento                     |
| some(func)     | Verifica se algum elemento satisfaz a condição |
| every(func)    | Verifica se todos os elementos satisfazem      |
| sort(func)     | Ordena array (modifica original)               |
| reverse()      | Inverte ordem (modifica original)              |
| includes(item) | Verifica se contém elemento                    |
| join(sep)      | Junta em string com separador                  |

Tabela 1: Métodos úteis de arrays

## 5 Procedimento

### 5.1 Tarefa 1: Aprender Objetos e Arrays de Objetos

#### 5.1.1 Passo 1: Experimentar na consola

Abra a consola do browser (F12) e experimente:

```
// Criar objeto de item
let item = {
  id: 1,
  nome: "Leite",
  quantidade: 2,
  categoria: "Laticínios"
};

console.log(item);
console.log(item.nome);
```

```
console.log(item['quantidade']);

// Modificar
item.quantidade = 3;
item.comprado = false;

console.log(item);
```

### 5.1.2 Passo 2: Trabalhar com Arrays de Objetos

```
// Criar objetos 1 a 1
let item1 = { id: 1, nome: "Leite", quantidade: 2, categoria: "Laticí  
nios" };
let item2 = { id: 2, nome: "Pão", quantidade: 1, categoria: "Padaria" };
let item3 = { id: 3, nome: "Maçã", quantidade: 5, categoria: "Frutas" };
let item4 = { id: 4, nome: "Queijo", quantidade: 1, categoria: "Laticí  
nios" };

// Criar array vazio e adicionar os objetos
let compras = [];
compras.push(item1);
compras.push(item2);
compras.push(item3);
compras.push(item4);

// Percorrer
compras.forEach(item => {
  console.log(item.nome + " - Qtd: " + item.quantidade);
});

// Filtrar (só laticínios)
let laticinios = compras.filter(item => item.categoria === "Laticínios")
;
console.log(laticinios);

// Mapear (só nomes e quantidades)
let resumo = compras.map(item => ({
  nome: item.nome,
  qtd: item.quantidade
}));
console.log(resumo);

// Encontrar total
let totalUnidades = compras.reduce((total, item) => total + item.  
quantidade, 0);
console.log("Total: " + totalUnidades);
```

## 5.2 Tarefa 2: Aprofundar Lista de Compras com Objetos

Vamos modificar o código do Lab 05 para usar objetos em vez de strings simples.

**Implementar por blocos pequenos**

Evite fazer copy-paste do bloco inteiro. Compare o novo código com o exemplo que já tem implementado e adicione as alterações por partes pequenas. Esta abordagem ajuda a compreender cada mudança e reduz erros de integração.

**5.2.1 Passo 1: Atualizar Estrutura de Dados**

Modifique o ficheiro `lista-compras.js`:

Atualize também o HTML da zona de inserção para permitir escolher a categoria do item:

```
<div class="adicionar-item">
  <input type="text" id="input-item" placeholder="Adicionar item...">
  <select id="select-categoria">
    <option value="Geral">Geral</option>
    <option value="Frutas">Frutas</option>
    <option value="Legumes">Legumes</option>
    <option value="Laticínios">Laticínios</option>
    <option value="Padaria">Padaria</option>
  </select>
  <button id="btn-adicionar">Adicionar</button>
</div>
```

```
// Array agora com objetos
let itens = [];
let proximoId = 1;

// Funcao para adicionar item
function adicionarItem(nome, categoria = "Geral") {
  let item = {
    id: proximoId++,
    nome: nome,
    quantidade: 1,
    categoria: categoria,
    comprado: false
  };

  itens.push(item);
  renderizarLista();
}

// Funcao para remover item
function removerItem(id) {
  itens = itens.filter(item => item.id !== id);
  renderizarLista();
}

// Funcao para alterar quantidade
function alterarQuantidade(id, delta) {
  let item = itens.find(item => item.id === id);
  if (item) {
    item.quantidade += delta;
    if (item.quantidade <= 0) {
      removerItem(id);
    } else {

```

```
        renderizarLista();
    }
}

// Funcao para marcar/desmarcar como comprado
function marcarComprado(id) {
    let item = itens.find(item => item.id === id);
    if (item) {
        item.comprado = !item.comprado;
        renderizarLista();
    }
}
```

No código que trata o clique do botão Adicionar, passe também a categoria selecionada:

```
let inputItem = document.getElementById('input-item');
let btnAdicionar = document.getElementById('btn-adicionar');
let selectCategoria = document.getElementById('select-categoria');

btnAdicionar.addEventListener('click', function() {
    let nome = inputItem.value.trim();
    let categoria = selectCategoria.value;

    if (nome === '') {
        alert('Por favor, insira um nome para o item!');
        return;
    }

    adicionarItem(nome, categoria);
    inputItem.value = '';
    selectCategoria.value = 'Geral';
    inputItem.focus();
});
```

## 5.2.2 Passo 2: Atualizar Renderização

Modifique a função renderizarLista:

```
function renderizarLista() {
    let listaElemento = document.getElementById('lista-itens');
    listaElemento.innerHTML = '';

    for (let indice = 0; indice < itens.length; indice++) {
        let item = itens[indice];

        let li = document.createElement('li');
        li.classList.add('item-lista');
        if (item.comprado) {
            li.classList.add('item-comprado');
        }

        // Checkbox
        let checkbox = document.createElement('input');
        checkbox.type = 'checkbox';
        checkbox.checked = item.comprado;
```

```
checkbox.addEventListener('change', () => marcarComprado(item.id));

// Nome e categoria
let span = document.createElement('span');
span.classList.add('item-nome');
span.textContent = item.nome + ' [' + item.categoria + ']';

// Quantidade
let qtdDiv = document.createElement('div');
qtdDiv.classList.add('quantidade-controlo');

let btnMenos = document.createElement('button');
btnMenos.textContent = '-';
btnMenos.addEventListener('click', () => alterarQuantidade(item.id, -1));

let qtdSpan = document.createElement('span');
qtdSpan.textContent = item.quantidade;

let btnMais = document.createElement('button');
btnMais.textContent = '+';
btnMais.addEventListener('click', () => alterarQuantidade(item.id, 1));

qtdDiv.appendChild(btnMenos);
qtdDiv.appendChild(qtdSpan);
qtdDiv.appendChild(btnMais);

// Botão remover
let btnRemover = document.createElement('button');
btnRemover.textContent = 'Remover';
btnRemover.classList.add('btn-remover');
btnRemover.onclick = () => removerItem(item.id);

// Adicionar tudo
li.appendChild(checkbox);
li.appendChild(span);
li.appendChild(qtdDiv);
li.appendChild(btnRemover);

listaElemento.appendChild(li);
}
```

### 5.2.3 Passo 3: Adicionar Estatísticas

Atualize o bloco de informação em `lista-compras.html` para mostrar os novos totais:

```
<div class="info">
  <p>Total de itens: <span id="total-itens">0</span></p>
  <p>Total de unidades: <span id="total-unidades">0</span></p>
  <p>Itens comprados: <span id="comprados">0</span></p>
</div>
```

Adicione também algum CSS para os novos elementos visuais:

```
#select-categoria {
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 4px;
}

.quantidade-controlo {
  display: flex;
  align-items: center;
  gap: 8px;
}

.quantidade-controlo button {
  padding: 4px 10px;
}

.item-comprado .item-nome {
  text-decoration: line-through;
  opacity: 0.7;
}
```

Altera a função `atualizarTotal` para `atualizarEstatisticas` (é um nome mais adequado) ficando assim:

```
function atualizarEstatisticas() {
  let totalItens = itens.length;
  let totalUnidades = itens.reduce((total, item) => total + item.
    quantidade, 0);
  let comprados = itens.filter(item => item.comprado).length;

  document.getElementById('total-itens').textContent = totalItens;
  document.getElementById('total-unidades').textContent =
    totalUnidades;
  document.getElementById('comprados').textContent = comprados;
}
```

### Ponto de situação da Tarefa 2

No final desta tarefa, a nova versão da lista de compras já deve estar funcional: adicionar itens com categoria, alterar quantidades, marcar como comprado, remover e ver estatísticas. No entanto, ainda existe uma limitação importante: ao fechar o browser, perde-se toda a informação.

Na tarefa seguinte vamos resolver exatamente essa limitação, adicionando persistência com `LocalStorage`.

## 5.3 Tarefa 3: Persistência com LocalStorage

Nesta tarefa vamos guardar os dados da aplicação no browser para que a lista não se perca entre sessões. O objetivo é simples: ao fechar e voltar a abrir a página, os itens devem continuar disponíveis.

### 5.3.1 Passo 1: Guardar e Carregar

```
// Guardar dados em LocalStorage
function guardarDados() {
    let dados = {
        itens: itens,
        proximoId: proximoId
    };
    localStorage.setItem('listaCompras', JSON.stringify(dados));
    console.log("Dados guardados!");
}

// Carregar dados do LocalStorage
function carregarDados() {
    let jsonSalvo = localStorage.getItem('listaCompras');
    if (jsonSalvo) {
        try {
            let dados = JSON.parse(jsonSalvo);
            itens = dados.itens;
            proximoId = dados.proximoId;

            renderizarLista();
            console.log("Dados carregados!");
        } catch (erro) {
            console.error("Erro ao carregar dados:", erro);
        }
    }
}

// Se o script estiver com atributo defer no HTML,
// basta chamar a inicialização no fim do ficheiro:
carregarDados();
```

Depois de criar a função `guardarDados()`, volte às funções da Tarefa 2 que alteram os dados e chame `guardarDados()` nesses pontos.

```
function adicionarItem(nome, categoria = "Geral") {
    // ... criar item e adicionar ao array
    itens.push(item);
    renderizarLista();
    guardarDados();
}

function removerItem(id) {
    itens = itens.filter(item => item.id !== id);
    renderizarLista();
    guardarDados();
}

function alterarQuantidade(id, delta) {
    let item = itens.find(item => item.id === id);
    if (item) {
        item.quantidade += delta;
        if (item.quantidade <= 0) {
            removerItem(id); // removerItem já guarda os dados
        } else {
            renderizarLista();
            guardarDados();
        }
    }
}
```

```
    }  
  }  
}  
  
function marcarComprado(id) {  
  let item = itens.find(item => item.id === id);  
  if (item) {  
    item.comprado = !item.comprado;  
    renderizarLista();  
    guardarDados();  
  }  
}
```

**Regra prática:** sempre que o array `itens` for alterado, deve chamar `guardarDados()` logo a seguir.

### Ordem de inicialização

Se usar `carregarDados()` no fim do ficheiro, coloque esta chamada apenas depois de declarar as funções e configurar os event listeners. Assim, a aplicação arranca com os dados guardados e com a interface pronta a responder.

## 5.4 Tarefa 4: Detalhes Finais

### 5.4.1 Ajustar Ordenação

No Lab 05 a ordenação já existia e era alfabética (A-Z), feita diretamente no botão de ordenar. Agora, com objetos, podemos implementar ordenações mais avançadas. Por exemplo, podemos ordenar por categoria ou por quantidade. Assim, crie as seguintes funções de ordenação:

```
// Já existia no Lab 05 (A-Z por nome)  
function ordenarPorNome() {  
  itens.sort((a, b) => a.nome.localeCompare(b.nome));  
  renderizarLista();  
}  
  
// Novo critério: categoria  
function ordenarPorCategoria() {  
  itens.sort((a, b) => a.categoria.localeCompare(b.categoria));  
  renderizarLista();  
}  
  
// Novo critério: quantidade (maior para menor)  
function ordenarPorQuantidade() {  
  itens.sort((a, b) => b.quantidade - a.quantidade);  
  renderizarLista();  
}
```

Adicione também os botões correspondentes no HTML para cada uma das ordenações, substituindo o botão de ordenar existente.

## 6 Resultado Esperado

No final deste laboratório, deve ter:

- ☐ Compreensão profunda de objetos em JavaScript
- ☐ Capacidade de trabalhar com arrays de objetos
- ☐ Conhecimento de JSON (stringify / parse)
- ☐ Lista de Compras com dados persistentes
- ☐ Capacidade de ordenar e processar dados
- ☐ Código bem organizado e documentado
- ☐ LocalStorage a funcionar corretamente

## 7 Questões para Reflexão

1. Qual a diferença entre um objeto e um array em JavaScript?
2. Porque é que JSON é importante?
3. Como funciona o método `reduce()` em arrays?
4. Qual a diferença entre `map()`, `filter()` e `forEach()`?
5. Como persiste dados no LocalStorage?
6. Qual a vantagem de usar IDs únicos em objetos?

## 8 Próximos Passos

No próximo laboratório (Lab 07), iremos trabalhar com:

- APIs e fetch
- Comunicação com servidores
- Promises e async/await
- Tratamento de erros
- Dados dinâmicos de fontes externas

### Recursos recomendados para estudo:

- MDN - Objects: [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Object](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object)
- MDN - JSON: [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/JSON](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON)
- MDN - Array Methods: [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Array](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array)
- MDN - LocalStorage: <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>

## 9 Referências

- **MDN - Objects:** [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Object](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object)
- **MDN - JSON:** [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/JSON](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON)
- **MDN - Array methods:** [https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global\\_Objects/Array](https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array)
- **MDN - LocalStorage:** <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- **JavaScript.info - Objects:** <https://javascript.info/object>
- **JavaScript.info - Arrays:** <https://javascript.info/array>